New Relic APM — Day 4 Study Guide: Custom Instrumentation, RUM and Synthetics

Oct 29, 2026 · @Vivek Arora

Day 4 at a glance

Days 1–3 used what the agents collect on their own. Day 4 is about filling the gaps: telling the agent about code and business context it cannot see, measuring what real users experience in their browsers, and checking the application from the outside before users notice it is broken.

Agenda

Time	Session	Lab
09:30 – 09:45	Day 3 recap; extend the lab system	—
09:45 – 10:45	**Module 10** — Custom instrumentation	—
10:45 – 11:00	Break	—
11:00 – 12:00	Lab 4.1 — Instrument the owner search: custom methods, attributes, events and naming	Lab 4.1 (60 min)
12:00 – 13:00	**Module 11** — Real user monitoring and browser insights	—
13:00 – 14:00	Lunch	—
14:00 – 15:00	Lab 4.2 — Enable RUM through the APM agent and read the user experience	Lab 4.2 (60 min)
15:00 – 15:15	Break	—
15:15 – 16:00	**Module 12** — Synthetic monitoring	—
16:00 – 17:15	Lab 4.3 — Ping, API and journey monitors from a private location, with SLA and alerts	Lab 4.3 (75 min)
17:15 – 17:30	Course wrap-up, knowledge check, Q&A	—

Learning objectives

By the end of Day 4 you can:

1. Decide when custom instrumentation is worth it, and pick the lightest tool: configuration, XML, annotations or the agent API.
2. Instrument custom methods in Java, .NET and Node.js, and see them as segments in traces and as metrics.
3. Add custom attributes and record custom events that carry business context, without leaking personal data or creating high-cardinality noise.
4. Name and segment transactions so the transaction list answers business questions — and avoid metric grouping issues.
5. Enable browser monitoring through the APM agent and read page load timings, Core Web Vitals, JavaScript errors and browser traces.
6. Use session traces, page views and the geography view to describe what users experience, where and on which devices.
7. Create ping, scripted API and scripted browser monitors, run them from public or private locations, and use them for SLA reporting and alerting.

How to use this guide with the lab guide

This study guide explains the concepts; every command, click and expected screen is in the separate **New Relic APM — Day 4 Lab Guide**. Each module ends with a *"Do Lab 4.x"* pointer. Screens in the lab guide are redrawn illustrations labelled **Expected view**; menu names may differ slightly in your account.

What the lab system gains today

Addition	Why	Lab
Code changes to Spring PetClinic — the `newrelic-api` dependency, a small telemetry class, an XML extension	Custom segments, attributes, a custom event and segmented transaction names for the owner search	4.1
Browser agent in PetClinic pages, inserted by the Java agent API	Page views, Core Web Vitals, JavaScript errors and browser traces linked to the backend	4.2
Headless browser load (Playwright container)	Real browsers run the browser agent; `curl` does not execute JavaScript, so it creates no page views	4.2

Addition	Why	Lab
Synthetics job manager (private location on the lab VM)	Monitors can reach the lab VM, which public locations on the internet cannot	4.3

The lab VM keeps easyTravel, PetClinic, PostgreSQL and the infrastructure agent from Days 1–3. The job manager needs about 3.3 GB of memory per heavyweight worker; the lab runs it with one worker.

Module 10 — Custom instrumentation

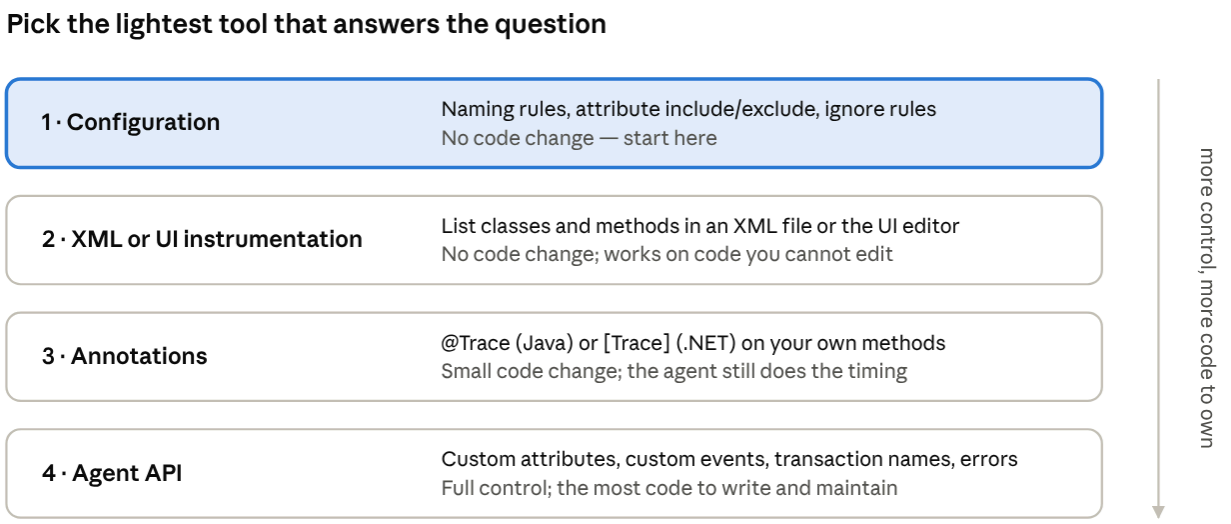
APM agents instrument the frameworks they know: servlet containers, web frameworks, JDBC, HTTP clients, messaging libraries. Everything else is a gap. A transaction trace then shows a long block of *application code* with no names inside, a background job never appears as a transaction, and nobody can answer "how many searches found nothing?" because the agent does not know what a search is.

10.1 Use cases for custom instrumentation

Gap	Symptom in New Relic	What to add
Your own slow methods	Trace shows one large *uninstrumented time* block	A custom method (segment)
Unsupported framework or a background job	No transaction at all, or a single catch-all name	A transaction start point (`dispatcher = true`, `[Transaction]`, `startBackgroundTransaction`)
Business context	You can see *that* a request was slow, not *which customer tier, plan or outcome*	Custom attributes
Business activity	Counts of searches, orders, payments by outcome are not in any event	A custom event
Poor transaction names	One name hides several behaviours, or thousands of names hide everything	Transaction naming

Gap	Symptom in New Relic	What to add
Handled errors	The code catches the exception, so the agent never sees it	`noticeError`

Every addition is code or configuration someone must maintain. Start with the question you need to answer, then choose the lightest tool that answers it.



custom instrumentation options · lightest first

10.2 Instrumenting custom methods

A *custom method* becomes a **segment** in transaction traces and distributed traces, and a metric (for example `Custom/Pricing/calculate`) you can chart and alert on. A *transaction start point* creates a new transaction when none is running — the way to see background jobs, queue consumers and scheduled tasks.

Java — no code change (XML extension). Put a file in the `extensions` folder next to `newrelic.jar` (or use **APM & services → your app → Settings → Instrumentation**). Restart the application so already-loaded classes are instrumented.

```
<?xml version="1.0" encoding="UTF-8"?>
<extension xmlns="https://newrelic.com/docs/java/xsd/v1.0"
           name="shop-pricing" version="1.0" enabled="true">
  <instrumentation metricPrefix="Custom">
    <pointcut transactionStartPoint="false">
      <className>com.example.shop.PriceCalculator</className>
      <method>
        <name>calculate</name>
      </method>
    </pointcut>
  </instrumentation>
</extension>
```

Java — annotations. Add the `com.newrelic.agent.java:newrelic-api` dependency (same version as the agent). The API calls do nothing when the agent is not attached, so the code is safe to ship everywhere.

```
import com.newrelic.api.agent.Trace;

public class PriceCalculator {

    @Trace(metricName = "Custom/Pricing/calculate")
    public Money calculate(Cart cart) { /* ... */ }

    // A scheduled job the agent cannot see: start a transaction here
    @Trace(dispatcher = true)
    public void runNightlyReconciliation() { /* ... */ }
}
```

.NET. The same two choices: an XML file in the agent's `extensions` folder, or attributes from the `NewRelic.Agent.Api` package.

```
<extension xmlns="urn:newrelic-extension">
  <instrumentation>
    <tracerFactory>
      <match assemblyName="Shop.Core" className="Shop.Core.PriceCalculator">
        <exactMethodMatcher methodName="Calculate" />
      </match>
    </tracerFactory>
  </instrumentation>
</extension>
```

```
using NewRelic.Api.Agent;

public class PriceCalculator
{
    [Trace]
    public Money Calculate(Cart cart) { /* ... */ }

    [Transaction]   // non-web transaction for a background job
    public void RunNightlyReconciliation() { /* ... */ }
}
```

Node.js. There are no annotations; wrap the work in the API. `require('newrelic')` must be the first line the application loads.

```
const newrelic = require('newrelic');

async function calculate(cart) {
  // a segment inside the current transaction
  return newrelic.startSegment('pricing/calculate', true, async () => {
    /* ... */
  });
}

// a background job as its own transaction; it ends when the promise settles
newrelic.startBackgroundTransaction('nightly-reconciliation', 'jobs', () =>
reconcile());
```

Instrument a handful of meaningful methods, not every method: each segment costs a little time and trace space, and very hot methods (called thousands of times per request) add overhead and noise.

10.3 Creating custom events and attributes

Custom attributes add context to data the agent already sends. Transaction attributes appear on `Transaction` and `TransactionError` events, in traces and in Errors Inbox; span attributes appear on one segment in a distributed trace. With `browser_monitoring.attributes.enabled`, chosen transaction attributes also reach the page view of that request.

Custom events are new rows of your own type, one per business occurrence — a search, an order, a payment — queryable with NRQL like any other event.

Task	Java	.NET	Node.js
Attribute on the transaction	`NewRelic.addCustomParameter("plan", "gold")`	`NewRelic.GetAgent().CurrentTransaction.AddCustomAttribute("plan", "gold")`	`newrelic.addCustomAttribute('plan', 'gold')`
Attribute on the current span	`NewRelic.getAgent().getTracedMethod().addCustomAttribute(...)`	`NewRelic.GetAgent().CurrentSpan.AddCustomAttribute(...)`	`newrelic.addCustomSpanAttribute(...)`
Custom event	`NewRelic.getAgent().getInsights().recordCustomEvent("OrderPlaced", map)`	`NewRelic.RecordCustomEvent("OrderPlaced", dict)`	`newrelic.recordCustomEvent('OrderPlaced', {...})`
Report a handled error	`NewRelic.noticeError(e)`	`NewRelic.NoticeError(ex)`	`newrelic.noticeError(err)`

Rules that keep custom data useful:

- **No personal data.** Never send names, e-mail addresses, card numbers or free text typed by users. Send what you can derive and need: an outcome, a count, a tier, a length.
- **Low cardinality where you FACET.** `plan` with five values is a good facet; `orderId` is a good *lookup* attribute but a useless facet.
- **Stable, namespaced keys.** `search.outcome`, `search.resultCount` — not `Outcome`, `result_count`, `searchOutcome` in three services.
- **Agents cap custom data.** Each agent limits the number and size of custom attributes and how many custom events it keeps per harvest; check the agent's configuration reference before recording an event per loop iteration.

```
-- Business question answered by a custom event
SELECT count(*) FROM OwnerSearch FACET outcome TIMESERIES SINCE 1 hour ago

-- Performance by business context, from a custom attribute
SELECT percentile(duration, 95) FROM Transaction
WHERE appName = 'petclinic' FACET search.outcome SINCE 1 hour ago
```

10.4 Transaction naming and segmentation

The agent names a web transaction from the framework route it recognises — a Spring controller mapping, a servlet name, a JAX-RS path. If it recognises nothing, it falls back to the URL, and every distinct URL (`/owners/1`, `/owners/2`, …) can become its own name: a **metric grouping issue**. New Relic then rolls names up into catch-all buckets and the transaction list stops being useful.

Good names have two properties:

- **One name per behaviour.** If one route does three different things — a search that finds nothing, one owner, or a page of owners — its average hides all three. Segmenting it into three names lets each have its own response time, Apdex and alert.
- **A small, fixed set.** Names come from a known list (outcomes, operations, routes with parameters removed), never from user input, IDs or timestamps.

```
// Segment the owner search by outcome: three names, not one per last name
NewRelic.setTransactionName("OwnerSearch", "/" + outcome);   // none | single
| many
```

.NET uses `NewRelic.SetTransactionName(category, name)`; Node.js `newrelic.setTransactionName(name)`. An API name wins over the framework name, so call it once you know the outcome. When you rename, update dashboards, alert conditions and key transactions that used the old name.

Attributes are the alternative to renaming: `search.outcome` on one transaction name lets you FACET by outcome without changing names. Rename when the behaviours need their own Apdex, alerts and trace thresholds; add an attribute when you only need to slice.

> **Do Lab 4.1 — Instrument the owner search** in the Day 4 Lab Guide (60 min): add an XML extension for a PetClinic method with no code change, then use the Java agent API to add a traced method, custom attributes, an `OwnerSearch` custom event and outcome-based transaction names — and find each one in traces, NRQL and the transaction list.

Module 11 — Real user monitoring and browser insights

APM measures the server: from the moment a request arrives until the response leaves. Users experience much more — DNS, network, waiting in queues, downloading scripts and images, the browser building and painting the page, and JavaScript running after it loads. A page can be slow or broken for users while every APM chart is green. Real user monitoring (RUM) measures that part, from every real browser that loads the page.

11.1 Enabling RUM via the APM agent

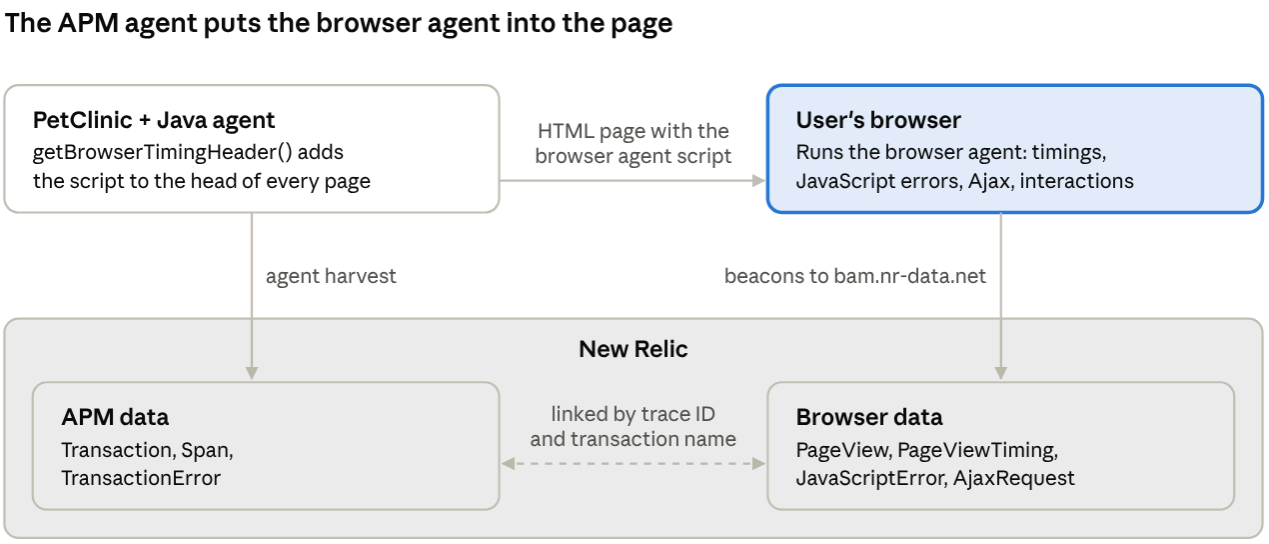
The browser agent is a JavaScript snippet in the `<head>` of each page. There are four ways to get it there:

Method	How	When
APM agent, automatic	The APM agent inserts the snippet into pages it renders	Supported server-side page frameworks (for the Java agent: compiled JSPs)
APM agent API	Your template calls the agent, e.g. Java `NewRelic.getBrowserTimingHeader()`	Template engines the agent does not inject into automatically (Thymeleaf, Velocity, …)
Copy/paste	Paste the snippet from the browser app settings into the HTML	Static sites, pages not rendered by an instrumented server
NPM package	Bundle `@newrelic/browser-agent` into the front-end build	Single-page apps built with a bundler

The two APM-agent methods have a big advantage: the snippet carries the APM transaction name and (optionally) chosen custom attributes, so a page view is tied to the server transaction that produced it, and the browser app shares the APM app's name. To use them:

1. In New Relic, add browser monitoring for the APM application and choose the **APM agent** method (rather than copy/paste). Pick the browser agent features — *Pro + SPA* collects the most.
2. Restart or reconnect the application so the agent receives the browser settings.
3. For frameworks without automatic injection, call the header API in the page template, as close to the top of `<head>` as possible but after the `<meta>` tags.

Only real browsers send RUM data. `curl`, load-test scripts that do not run JavaScript, and health checks produce APM transactions but no page views.



browser monitoring through the APM agent · data flow

11.2 Page load timings, JavaScript errors and browser traces

Page load timing splits each page view (`PageView` event) into phases:

Phase	Attribute	What it measures
Request queuing	`queueDuration`	Time before the app server picked up the request (when a front-end proxy sets the queue header)
Web application	`backendDuration`	Server time — the APM transaction
Network	`networkDuration`	DNS, connect, request and response transfer
DOM processing	`domProcessingDuration`	Parsing HTML and running scripts until the DOM is ready
Page rendering	`pageRenderingDuration`	From DOM ready to the load event
Total	`duration`	The whole page load

Core Web Vitals describe what users feel, and arrive as `PageViewTiming` events: *Largest Contentful Paint* (LCP, how fast the main content appears), *Interaction to Next Paint* (INP, how fast the page reacts to input) and *Cumulative Layout Shift* (CLS, how much the layout jumps). They are judged at the 75th percentile of page loads; Google's "good" thresholds are LCP up to 2.5 s, INP up to 200 ms and CLS up to 0.1.

JavaScript errors (`JavaScriptError` events) are uncaught errors in the page — a missing function, a failed script, an exception in a click handler. They appear in the browser app's JS errors view and in Errors Inbox, grouped like back-end errors, with the browser, page and stack trace. The server cannot see them: APM shows a successful request while the user sees a broken page.

Browser traces. With distributed tracing enabled in the browser app, the browser agent adds trace headers to Ajax and fetch calls to your own origins. The request then appears in one distributed trace from the browser span to the back-end spans and database calls, so a slow click can be followed into the server.

11.3 Session traces and user experience

A **session trace** is a timeline of one page visit: resource downloads, script and render work, user events, Ajax calls and errors, in order. It answers "what was the browser doing during those six seconds?" in a way aggregate charts cannot. New Relic records traces for a sample of visits; where *session replay* is enabled, a privacy-masked recording of the visit can also be played back.

User experience is the combination: Apdex for page loads, Core Web Vitals, JavaScript error rate, and the timings of the interactions that matter (search, checkout). Read them per page and per device — a page can be fine on desktop and poor on a mid-range phone.

11.4 Page views and the user geography heatmap

Page views list pages (grouped by the transaction name they came from) with throughput, average load time and their share of total user time — the page to optimise is often the one users spend most time waiting for, not the slowest one.

The **geography** view colours a world map by a metric (load time, LCP, page views, Apdex) per country or region, from the client IP of each page view. It shows whether a slow experience is everywhere (an application or front-end problem) or regional (CDN, network or data-centre distance).

```
-- Where users are and how fast their pages load
SELECT count(*), percentile(duration, 75) AS 'p75 load (s)' FROM PageView
WHERE appName = 'petclinic' FACET countryCode, city SINCE 1 day ago LIMIT 20

-- Core Web Vitals per page, the way Google scores them
SELECT percentile(largestContentfulPaint, 75) AS 'LCP p75 (s)' FROM
PageViewTiming
WHERE appName = 'petclinic' AND timingName = 'largestContentfulPaint' FACET
pageUrl SINCE 1 day ago

-- Which JavaScript errors, on which pages and browsers
SELECT count(*) FROM JavaScriptError WHERE appName = 'petclinic'
FACET errorClass, errorMessage, requestUri, userAgentName SINCE 1 day ago
```

In a training lab all browsers run in one place, so the map shows one or two locations. In production it is often the first chart that separates "the site is slow" from "the site is slow in one region".

> **Do Lab 4.2 — Enable RUM through the APM agent** in the Day 4 Lab Guide (60 min): enable browser monitoring for PetClinic with the APM agent method, insert the header through the Java agent API, generate real browser traffic with headless browsers, plant a JavaScript error, and read page load timings, Core Web Vitals, JS errors, browser-to-backend traces, session traces and the geography view.

Module 12 — Synthetic monitoring

RUM tells you what happened to the users who came. Synthetic monitoring tells you what *would* happen to a user right now, every minute, from chosen places — including at 03:00 when nobody is visiting, before a launch, and for a journey so important (login, checkout) that you want a check even when real traffic hides the problem.

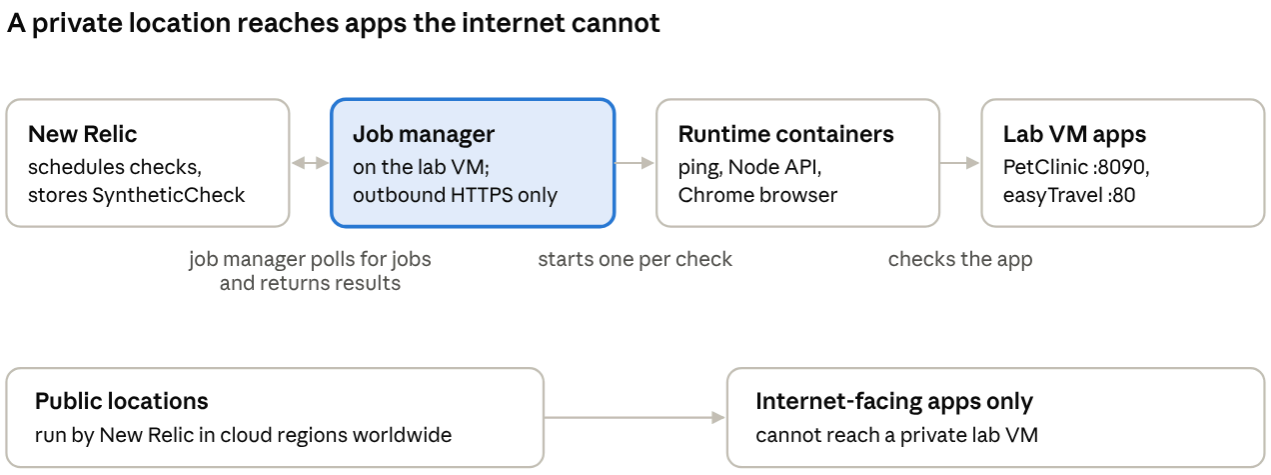
12.1 Creating synthetic monitors: ping, script and API test

Monitor type	What it does	Runs in	Use it for
Ping (availability)	One HTTP request; optional text validation	Lightweight ping runtime	Is the site up? Health endpoints
Simple browser	Loads one URL in a real Chrome browser	Browser runtime	Is the page rendering, with all its resources?

Monitor type	What it does	Runs in	Use it for
Scripted browser	A Selenium WebDriver script: navigate, click, type, assert	Browser runtime	Multi-step user journeys
Step monitor	The same journey built from steps in the UI, without code	Browser runtime	Journeys owned by people who do not write scripts
Scripted API	A Node.js script with an HTTP client and assertions	Node API runtime	API contracts, multi-call API flows, auth tokens
Certificate check, broken links	TLS certificate expiry; dead links on a page	—	Hygiene checks

Each monitor has a **frequency** (1 minute to 1 day), one or more **locations**, and per run produces a `SyntheticCheck` event (result, duration, location, error) plus `SyntheticRequest` events for the requests it made.

Locations. Public locations are run by New Relic in cloud regions around the world; they can only reach applications on the internet. A **private location** is a *synthetics job manager* you run (Docker, Podman or Kubernetes) inside your network. It needs only outbound HTTPS to New Relic: it polls for jobs, starts a runtime container for each check, and sends the result back. Size it for its workload — the job manager documentation asks for about one CPU core and 3.3 GB of memory per heavyweight (browser or scripted API) worker.



public and private synthetics locations

A scripted API test (Node.js API runtime):

```
const assert = require('assert');
const BASE = 'https://shop.example.com';

$http.get(BASE + '/api/health', function (err, response, body) {
  assert.ifError(err);
  assert.equal(response.statusCode, 200, 'health endpoint should answer
200');
  const health = typeof body === 'string' ? JSON.parse(body) : body;
  assert.equal(health.status, 'UP', 'service should report UP');
});
```

Secrets (API keys, test-user passwords) go into **secure credentials** and are read as `$secure.NAME`; they are masked in results and never written into the script.

12.2 Simulating user journeys and transactions

A scripted browser monitor drives Chrome through the journey a customer takes:

```
const assert = require('assert');
const By = $selenium.By, until = $selenium.until;

await $webDriver.get('https://shop.example.com/');
await $webDriver.findElement(By.id('search')).sendKeys('running shoes');
await $webDriver.findElement(By.css('button[type="submit"]')).click();
const results = await $webDriver.wait(until.elementLocated(By.id('results')),
10000);
assert.ok((await results.getText()).includes('running'), 'search should show
results');
```

Good journey scripts:

- **Wait for elements, never sleep.** `until.elementLocated` with a timeout tolerates normal variation; fixed sleeps make monitors slow and flaky.
- **Assert the outcome a user cares about** (results shown, order confirmed), not only that a page loaded — a 200 response with an error message inside is still a failure.
- **Use test data and test accounts** that cannot place real orders or send real e-mails, and mark synthetic traffic so it can be excluded from business reports.
- **Stay inside the limits**: a browser script stops after three minutes; keep one journey per monitor so a failure points at one flow.

Synthetic requests carry New Relic headers, so the APM transactions they cause are tagged and linked: from a failed check you can open the back-end transaction and its trace, and in NRQL `Transaction` events from synthetic checks have `nr.syntheticsMonitorId` set.

12.3 SLA monitoring with synthetics

Synthetic checks are a controlled, constant sample — the same request from the same places at a fixed rate — which makes them a fair basis for availability SLAs: busy hours do not outweigh quiet ones, and 03:00 outages count.

- **Availability** = successful checks ÷ all checks, per monitor (and per location) over the SLA period.
- **Latency SLA** = share of checks below a duration target, or a percentile of check duration.
- The **SLA report** in synthetic monitoring shows daily, weekly and monthly availability and Apdex per monitor; NRQL gives the same numbers for dashboards (Day 3).

```
SELECT percentage(count(*), WHERE result = 'SUCCESS') AS 'availability %',
       percentile(duration, 95) / 1000 AS 'p95 (s)'
FROM SyntheticCheck FACET monitorName SINCE 30 days ago
```

Check frequency and location count set the resolution: one location every 5 minutes is 288 checks a day, and one failed check moves a daily figure by about 0.35 points. State the monitor, frequency and locations next to any SLA figure.

12.4 Using synthetics with alerting

- **Failure conditions.** Alert when checks fail — for several locations, prefer a *multi-location* condition (critical when two or more locations fail at once) so one flaky region does not page anyone. With one location, require two consecutive failures.
- **NRQL conditions** cover everything else: duration above a target, a custom value saved with `$util.insights.set`, or failures across a group of monitors.
- **Signal loss** catches the monitor that silently stops running — for a private location, the job manager being down looks exactly like that.
- **Maintenance.** Mute the synthetics conditions (Day 2 muting rules) or disable monitors during planned work; synthetics never know a deployment is planned.
- **Route like any other alert**: into the service's policy and workflow, with the runbook link and the monitor's result URL in the notification, so the on-call engineer can see the failed step, its screenshot and the linked back-end trace.

> **Do Lab 4.3 — Synthetic monitoring from a private location** in the Day 4 Lab Guide (75 min): run a synthetics job manager on the lab VM, create ping, scripted API and scripted browser monitors for PetClinic and easyTravel, read results, linked back-end traces and the SLA report, then alert on journey failures and on the private location going silent — and rehearse an outage.

User-experience runbook and Day 4 cheat sheets

"Users say the site is slow or broken" — an outside-in runbook

Step	Look at	If you see…	Then…
1	**Synthetics**: are the journey monitors failing, from which locations?	All locations fail	Outage or broken release: go to APM (Day 1) and Errors Inbox (Day 2)
		One location fails	Regional network, DNS or CDN: check the geography view (step 3)
2	**Browser summary**: page load, Core Web Vitals, JS error rate	Page load up, *web application* phase up	Server problem: APM transactions, traces, database (Days 1 and 3)
		Page load up, *network* or *rendering* up, server flat	Front-end or network: large assets, third-party scripts, CDN
		JS errors up, page load flat	Front-end bug: JS errors view, stack trace, release that introduced it
3	**Geography** and **device / browser** facets	One region or one device class	Regional infrastructure, or a browser-specific bug
4	**Session traces** for slow or failing visits	Long gaps, blocking scripts, failed Ajax	The exact resource or call to fix
5	**Browser trace** (distributed tracing) for a slow Ajax call	Time in back-end spans	Continue in APM with the trace ID
6	**Custom events and attributes**	One business outcome or customer tier affected	Scope the impact in business terms for the incident update
7	Verify	Synthetic checks green, Core Web Vitals and error rate back to baseline	Close the incident; add a monitor if the failure was not caught

Agent API cheat sheet — Module 10

Task	Java (`com.newrelic.api.agent`)	.NET (`NewRelic.Api.Agent`)	Node.js (`newrelic`)
Time a method	`@Trace`	`[Trace]`	`newrelic.startSegment(name, true, fn)`
Start a transaction	`@Trace(dispatcher = true)`	`[Transaction]`	`newrelic.startBackgroundTransaction(name, group, fn)` / `startWebTransaction(url, fn)`
No-code method	XML in `extensions/` or the UI editor	XML in the agent's `extensions` folder	— (use the API)
Attribute on transaction	`NewRelic.addCustomParameter(k, v)`	`CurrentTransaction.AddCustomAttribute(k, v)`	`newrelic.addCustomAttribute(k, v)`
Attribute on span	`getTracedMethod().addCustomAttribute(k, v)`	`CurrentSpan.AddCustomAttribute(k, v)`	`newrelic.addCustomSpanAttribute(k, v)`
Custom event	`getInsights().recordCustomEvent(type, map)`	`NewRelic.RecordCustomEvent(type, attrs)`	`newrelic.recordCustomEvent(type, attrs)`
Transaction name	`NewRelic.setTransactionName(category, name)`	`NewRelic.SetTransactionName(category, name)`	`newrelic.setTransactionName(name)`
Handled error	`NewRelic.noticeError(e)`	`NewRelic.NoticeError(ex)`	`newrelic.noticeError(err)`
Skip a transaction	`NewRelic.ignoreTransaction()`	`NewRelic.IgnoreTransaction()`	`newrelic.getTransaction().ignore()`

NRQL cheat sheet — custom data (Module 10)

```
-- Which custom event types exist, and their attributes
SHOW EVENT TYPES SINCE 1 day ago
SELECT keyset() FROM OwnerSearch SINCE 1 hour ago

-- Business outcome over time
SELECT count(*) FROM OwnerSearch FACET outcome TIMESERIES SINCE 3 hours ago

-- Performance per business context (custom attribute)
SELECT percentile(duration, 95), count(*) FROM Transaction
WHERE appName = 'petclinic' FACET search.outcome SINCE 1 hour ago

-- Custom segment timing (method instrumented with @Trace or XML)
SELECT average(newrelic.timeslice.value) * 1000 AS 'ms' FROM Metric
WHERE appName = 'petclinic' AND metricTimesliceName LIKE 'Custom/%'
FACET metricTimesliceName TIMESERIES SINCE 1 hour ago

-- Metric grouping check: how many transaction names?
SELECT uniqueCount(name) FROM Transaction WHERE appName = 'petclinic' SINCE 1
day ago
```

NRQL cheat sheet — browser (Module 11)

```
-- Page load phases per page
SELECT average(backendDuration), average(networkDuration),
average(domProcessingDuration),
       average(pageRenderingDuration), average(duration)
FROM PageView WHERE appName = 'petclinic' FACET pageUrl SINCE 1 hour ago

-- Core Web Vitals at p75
SELECT percentile(largestContentfulPaint, 75) FROM PageViewTiming
WHERE appName = 'petclinic' AND timingName = 'largestContentfulPaint' SINCE 1
day ago
SELECT percentile(interactionToNextPaint, 75) FROM PageViewTiming
WHERE appName = 'petclinic' AND timingName = 'interactionToNextPaint' SINCE 1
day ago

-- JS error rate: errors per 100 page views
SELECT filter(count(*), WHERE eventType() = 'JavaScriptError') /
       filter(count(*), WHERE eventType() = 'PageView') * 100 AS 'errors per
100 views'
FROM JavaScriptError, PageView WHERE appName = 'petclinic' TIMESERIES SINCE 3
hours ago

-- Users by device and browser
SELECT count(*) FROM PageView WHERE appName = 'petclinic'
FACET deviceType, userAgentName SINCE 1 day ago
```

NRQL cheat sheet — synthetics (Module 12)

```
-- Current state of every monitor
SELECT latest(result), latest(duration) / 1000 AS 'last run (s)'
FROM SyntheticCheck FACET monitorName, locationLabel SINCE 30 minutes ago

-- Failures and why
SELECT count(*) FROM SyntheticCheck WHERE result = 'FAILED'
FACET monitorName, error SINCE 1 day ago

-- Availability SLA per monitor
SELECT percentage(count(*), WHERE result = 'SUCCESS') FROM SyntheticCheck
FACET monitorName SINCE 30 days ago

-- Back-end transactions caused by synthetic checks
SELECT count(*), average(duration) FROM Transaction
WHERE nr.syntheticsMonitorId IS NOT NULL FACET appName, name SINCE 1 hour ago
```

Knowledge check, answers and sources

Questions

1. A trace shows 900 ms of *application code* with no segments inside. You cannot change the application's source. What do you use?
2. A nightly job runs inside the same JVM as the web application but never appears in APM. What is missing, and how do you add it in Java, .NET and Node.js?
3. A developer wants to add the customer's e-mail address as a custom attribute "to make support easier". What do you suggest instead?
4. What is the difference between a custom attribute and a custom event? Give one question each answers best.
5. An app reports 40,000 different transaction names, most like `/orders/12345`. What is this called, what does New Relic do about it, and how do you fix it at the source?
6. When should you segment a route into several transaction names rather than add an attribute?
7. Your Java application uses Thymeleaf templates. Why does the browser agent not appear in pages automatically, and how do you add it through the APM agent?
8. APM shows the checkout page answering in 180 ms, yet users complain it is slow. Which page load phases and browser metrics do you check?
9. A load test with `curl` runs for an hour, but the browser app shows no page views. Why?
10. Name three Core Web Vitals, what each measures, and the percentile at which they are judged.
11. Your application runs on an internal network. Can public synthetics locations monitor it? What do you deploy instead, and what network access does it need?
12. Which monitor type fits each case: (a) is the health endpoint up, (b) can a user search and open a result, (c) does the order API return a valid token and order?
13. A journey monitor running from one location fails once at 02:13 and passes at 02:14. How should the alert condition be set so this does not page anyone, while a real outage still does?
14. Why is synthetic availability a better basis for an SLA figure than the error rate of real traffic, and what must you state next to the figure?

Answers

1. **XML (or UI) custom instrumentation**: list the classes and methods in an extension file in the agent's `extensions` folder and restart. No code change; the methods become segments and `Custom/…` metrics.

2. A **transaction start point**. Java: `@Trace(dispatcher = true)` (or `transactionStartPoint="true"` in XML); .NET: `[Transaction]`; Node.js: `newrelic.startBackgroundTransaction(name, group, fn)`.
3. No personal data in telemetry. Record what support needs without identifying the person: a customer ID that support can look up in its own system, the customer tier, or the outcome — and check the organisation's data policy.
4. An **attribute** adds context to an event the agent already sends (a transaction, span or page view) — *"is p95 worse for gold customers?"*. A **custom event** is a new row per business occurrence — *"how many searches found nothing per hour?"*, even when no transaction is slow or failing.
5. A **metric grouping issue**. New Relic applies normalisation rules and rolls names into catch-all buckets, so the transaction list loses meaning. Fix it by naming from the route template (framework instrumentation, API naming with the parameters removed), never from IDs.
6. When the behaviours need their **own** Apdex, alert thresholds, trace thresholds and transaction-list rows — for example a search that finds nothing versus one that returns a page of results. If you only need to slice charts, an attribute is enough.
7. The Java agent injects automatically only into compiled JSPs. For Thymeleaf, enable browser monitoring for the APM app with the **APM agent** method and output `NewRelic.getBrowserTimingHeader()` near the top of `<head>` in the layout template.
8. Page load phases (`networkDuration`, `domProcessingDuration`, `pageRenderingDuration` against `backendDuration`), Core Web Vitals (LCP, INP, CLS), JavaScript errors, slow Ajax calls and session traces of slow visits — per device and geography.
9. `curl` does not run JavaScript, so the browser agent never executes. It creates APM transactions but no page views.
10. **LCP** — how quickly the main content appears; **INP** — how quickly the page responds to input; **CLS** — how much the layout moves unexpectedly. All are judged at the **75th percentile** of page loads.
11. No — public locations run on the internet. Deploy a **private location** (synthetics job manager) inside the network. It needs outbound HTTPS to New Relic only; no inbound access.
12. (a) **Ping**, (b) **scripted browser** (or a step monitor), (c) **scripted API**.
13. Require **two consecutive failures** (or, with several locations, a multi-location condition requiring at least two failing locations). Add signal-loss detection so a stopped monitor or job manager also alerts.

14. Checks are a fixed sample — same request, same places, same rate — so quiet hours count as much as busy ones and outages without traffic are still seen. State the monitor, its frequency and its locations; they set the resolution of the figure.

Sources

- New Relic docs — Guide to using the Java agent API and Java XML instrumentation examples
- New Relic docs — Instrument browser monitoring with the Java agent API and Java agent browser monitoring settings
- New Relic docs — .NET agent API
- New Relic docs — Install the synthetics job manager and Job manager configuration
- New Relic docs — Write synthetic API tests and Introduction to scripted browser monitors
- Google — Web Vitals

Agent APIs, UI labels and synthetics runtimes change between versions; check the agent's own API reference for the exact signatures in your version.